



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
IIC2233 — PROGRAMACIÓN AVANZADA 2026-1

Interrogación 2

28 de Mayo 2026

Instrucciones

- La evaluación consta de dos partes: 5 preguntas de selección múltiple y 4 preguntas de desarrollo.
- La interrogación consta de una sección de alternativas y una sección de desarrollo. La sección de desarrollo está compuesta por 4 preguntas. Tanto la sección de alternativas como cada una de las preguntas de desarrollo valen un 20 % de la nota final.
- Recibirás una hoja de respuestas que deberás rellenar con tus datos y las respuestas correspondientes a cada alternativa o pregunta de desarrollo. Sólo se corregirá la hoja de respuestas. **Ten mucha precaución de anotar correctamente tus datos.**
- Cada pregunta de selección múltiple tiene únicamente una alternativa correcta. Marcar dos o más alternativas invalidará la pregunta, considerándose incorrecta. Todas las preguntas tienen igual puntaje y no existe penalización por respuestas incorrectas. Se permite justificar la alternativa seleccionada en la hoja de respuesta; esta justificación será considerada en caso de corrección de la pregunta.
- Debes completar con tus datos personales en cada hoja utilizada. Luego debes entregar de vuelta **todas** las hojas de respuesta (tanto para alternativas como de desarrollo) de la prueba, aunque estas se encuentren en blanco.
- Solo podrás retirarte de la sala una vez hayan transcurrido 60 minutos desde que inició de la evaluación.
- Durante la evaluación se realizarán 2 rondas de preguntas. Estas preguntas únicamente serán respondidas por los profesores del ramo.
- En caso de que sea necesario, podrás solicitar hojas blancas para apoyar al desarrollo de la prueba. Para esto levanta la mano y espera que un ayudante se acerque a tu puesto.

Selección múltiple [20 % nota final]

1. ¿Cuál es la diferencia entre **conurrencia** y **paralelismo**?
 - A) Son sinónimos: ambos significan ejecutar varias tareas exactamente al mismo tiempo.
 - B) El paralelismo ejecuta varias tareas simultáneamente en distintos núcleos, mientras que la conurrencia gestiona varias tareas de forma intercalada, pudiendo ocurrir en un solo núcleo.
 - C) La conurrencia requiere siempre múltiples núcleos; mientras que el paralelismo funciona con uno solo.
 - D) El paralelismo solo se logra con **threading**, mientras que la conurrencia requiere **multiprocessing**.
 - E) La conurrencia es exclusiva de Python, mientras que el paralelismo existe en cualquier lenguaje.
2. Dado el siguiente código:

```
class Escritor(threading.Thread):  
    var_lock = threading.Lock()  
  
    def __init__(self, archivo):  
        super().__init__()  
        self.archivo = archivo  
  
    def run(self):  
        with self.var_lock:  
            self.archivo.write("línea\n")
```

¿Por qué `var_lock` está definido como atributo de clase?

- A) Para que cada instancia de `Escritor` tenga su propio `Lock` independiente.
- B) Porque un `Lock` definido dentro de `__init__` no puede usarse junto a la sentencia `with`.
- C) Porque los atributos de clase se adquieren y liberan automáticamente al entrar y salir de `run()`.
- D) Para que todas las instancias de `Escritor` compartan el mismo `Lock`, garantizando exclusión mutua sobre el archivo.
- E) Porque definir el `Lock` como atributo de instancia provocaría un `RuntimeError` al llamar a `start()`.

3. Dado el siguiente código.

```
1  import threading
2  import time
3
4  senal_a = threading.Event()
5  senal_b = threading.Event()
6
7  def thread_a():
8      time.sleep(5)
9      print("A-listo")
10     senal_b.wait()
11     time.sleep(1)
12     print("A-continuo")
13     senal_a.set()
14     time.sleep(1)
15     print("A-termino")
16
17 t_a = threading.Thread(
18     target=thread_a
19 )
```

```
20
21 def thread_b():
22     time.sleep(2)
23     print("B-listo")
24     senal_b.set()
25     time.sleep(1)
26     print("B-continuo")
27     senal_a.wait()
28     time.sleep(5)
29     print("B-termino")
30
31 t_b = threading.Thread(
32     target=thread_b
33 )
34 t_a.start()
35 t_b.start()
36 t_a.join()
37 t_b.join()
```

¿Cuál de las siguientes alternativas muestra la salida de este programa? Suponga que en la salida hemos reemplazado los saltos de línea por espacios.

- A) B-listo B-continuo A-listo A-continuo A-termino B-termino
- B) B-listo B-continuo A-listo A-continuo B-termino A-termino
- C) B-listo B-continuo *(el programa se bloquea y A nunca imprime nada.)*
- D) B-listo A-listo A-continuo A-termino B-continuo B-termino
- E) A-listo A-continuo A-termino B-listo B-continuo B-termino

4. Dado el siguiente código. Dos servidores registran la cantidad de mensajes procesados sobre su propio contador. Cada servidor lanza cuatro threads en simultáneo que aumentan ese contador un millón de veces cada uno.

```
1 import threading
2
3 N_WORKERS = 4
4 N_MENSAJES = 1_000_000
5
6 class ServidorNorte:
7     def __init__(self):
8         self.contador = 0
9         self.lock = threading.Lock()
10
11 class ServidorSur:
12     def __init__(self):
13         self.contador = 0
14
15 def trabajar_norte(servidor):
16     for _ in range(N_MENSAJES):
17         with servidor.lock:
18             servidor.contador += 1
19
20 def trabajar_sur(servidor):
21     for _ in range(N_MENSAJES):
22         servidor.contador += 1
23
24 servidor_norte = ServidorNorte()
25 servidor_sur = ServidorSur()
```

```
26 threads = []
27 for _ in range(N_WORKERS):
28     threads.append(threading.Thread(
29         target=trabajar_norte,
30         args=(servidor_norte,)
31     ))
32 for _ in range(N_WORKERS):
33     threads.append(threading.Thread(
34         target=trabajar_sur,
35         args=(servidor_sur,)
36     ))
37
38 for t in threads:
39     t.start()
40 for t in threads:
41     t.join()
42
43 print(
44     "ServidorNorte:",
45     servidor_norte.contador
46 )
47 print(
48     "ServidorSur:",
49     servidor_sur.contador
50 )
```

¿Qué relación se cumple entre los valores impresos?

- A) Ambos imprimen 4000000, porque cada servidor tiene cuatro *threads* que suman un millón cada uno.
- B) **ServidorSur** imprime un valor mayor que **ServidorNorte**, porque sus *threads* avanzan sin esperar y alcanzan a sumar más veces.
- C) **ServidorNorte** imprime 4000000, mientras que **ServidorSur** puede imprimir un valor menor, porque sus *threads* no están sincronizados.
- D) Ambos imprimen el mismo valor, aunque no necesariamente 4000000, porque comparten el mismo *scheduler* del sistema operativo.
- E) **ServidorNorte** imprime un valor menor que **ServidorSur**, porque algunos de sus *threads* quedan bloqueados y no alcanzan a sumar.

5. Los dos fragmentos siguientes crean e inician dos threads para ejecutarse en simultáneo. Al ejecutarlos, puede ocurrir que ninguna línea se imprima y el programa quede detenido.

Fragmento I

```
1 import threading
2 import time
3
4 lock_a = threading.Lock()
5 lock_b = threading.Lock()
6
7 def tarea_1():
8     with lock_a:
9         time.sleep(0.1)
10        with lock_b:
11            print("Tarea 1 lista")
12
13 def tarea_2():
14     with lock_b:
15         time.sleep(0.1)
16        with lock_a:
17            print("Tarea 2 lista")
18
19 t1 = threading.Thread(target=tarea_1)
20 t2 = threading.Thread(target=tarea_2)
21 threads = [t.start() for t in (t1, t2)]
```

Fragmento II

```
1 import threading
2
3 evento_a = threading.Event()
4 evento_b = threading.Event()
5
6 def tarea_3():
7     evento_b.wait()
8     evento_a.set()
9     print("Tarea 3 lista")
10
11 def tarea_4():
12     evento_a.wait()
13     evento_b.set()
14     print("Tarea 4 lista")
15
16 t3 = threading.Thread(target=tarea_3)
17 t4 = threading.Thread(target=tarea_4)
18 threads = [t.start() for t in (t3, t4)]
```

Considera las siguientes afirmaciones sobre los dos fragmentos:

1. El programa del Fragmento II termina correctamente si `tarea_3` y `tarea_4` llaman a `wait()` sobre la misma señal en lugar de sobre señales distintas.
2. En el Fragmento I, si modificamos el código para que `tarea_1` y `tarea_2` adquieran los `Lock` siempre en el mismo orden, entonces el programa termina sin quedar detenido.
3. En ambos fragmentos los *threads* quedan en una espera circular: cada *thread* necesita un recurso que el otro mantiene retenido o que el otro aún no ha liberado.
4. En el Fragmento I, el programa queda detenido porque `time.sleep(0.1)` se ejecuta dentro de la sección crítica. Mover esa pausa fuera de los bloques `with` hace que el programa termine.
5. En ambos fragmentos, el programa queda detenido porque los `Lock` y los `Event` están definidos como variables globales; declararlos dentro de cada función permite que el programa termine.
6. En el Fragmento II, el programa termina correctamente si al menos uno de los *threads* llama a `set()` antes de quedar esperando con `wait()`.

¿Cuáles afirmaciones son correctas?

- A) 2, 3 y 6
- B) 1, 2 y 3
- C) 4, 5 y 6
- D) 2, 4 y 5
- E) 1, 3 y 6

Preguntas Desarrollo

Pregunta 1. [20 % nota final]

Se desea diseñar una aplicación de escritorio calendario en PyQt que maneje eventos de un calendario. La aplicación tiene múltiples ventanas que están siempre abiertas, interactúan entre sí y se comunican a través de señales. Los componentes son:

- **class VentanaSemanal(QWidget):** Ventana que muestra los 7 días de la semana y en cada uno muestra el número de eventos de ese día. Permite seleccionar un día de la semana haciéndole click.
- **class VentanaDiaria(QWidget):** Ventana que muestra los detalles de los eventos del día seleccionado. Por ejemplo: si se selecciona el día lunes en la VentanaSemanal, esta ventana muestra los eventos del lunes.
- **class VentanaFormulario(QWidget):** Esta ventana permite crear un evento ingresando fecha, hora y descripción. El nuevo evento aparece en las otras ventanas luego de ser creado.
- **class Controlador(QObject):** Clase que centraliza la lógica y coordina la comunicación entre las ventanas. Actúa como *backend* del programa

El diagrama muestra la interfaz de usuario de la aplicación. Hay tres ventanas principales:

- VentanaSemanal:** Una ventana horizontal que muestra los días de la semana con el número de eventos para cada uno: Lunes (1), Martes (4), Miércoles (2), Jueves (5), Viernes (0), Sábado (3) y Domingo (1). El día Miércoles está seleccionado y resaltado.
- VentanaDiaria (Miércoles):** Una ventana que muestra los eventos para el día seleccionado. En este caso, muestra "Almuerzo oficina 1pm" y "Reunión directorio 3pm".
- VentanaFormulario:** Una ventana para crear un nuevo evento. Incluye un menú desplegable para "Día" (actualmente muestra "(Seleccione)"), campos de entrada para "Hora" (13:45) y un campo de texto para "Descripción". Hay un botón "Crear evento" a la derecha.

Consideraciones:

- Considere el modelo *backend/frontend* y los principios de alta cohesión y bajo acoplamiento.
- Para cada señal pedida indique qué clase la emite y la recibe, y qué información transmite.
- No escribas código.
- Enfóquese en la claridad del flujo de información.

A partir de lo anterior, responde las siguientes preguntas:

- (2 puntos)** Al hacer click en un día de la semana en VentanaSemanal, describa todas las señales emitidas entre componentes del programa y qué lógica se implementa en cada componente involucrado.
- (3 puntos)** Al rellenar el formulario en VentanaFormulario y hacer click en el botón "Crear evento", describa todas las señales emitidas entre componentes del programa y qué lógica se implementa en cada componente involucrado. Considere el caso en que el día del evento creado coincida con el día seleccionado.
- (1 punto)** ¿Qué información debe guardar la clase Controlador para poder manejar a las ventanas y la lógica del programa?

Pregunta 2. [20 % nota final]

Parte A — Análisis de casos (3 puntos)

Para cada caso, escoge únicamente una de las siguientes estructuras: **if/else** o **try/except**. Luego, justifica tu decisión en un máximo de 4 líneas. **Importante:** En los casos de **try/except**, no basta con mencionar que “puede ocurrir un error” o “una excepción”.

- **Caso 1:** En el inicio de sesión de un programa, la contraseña ingresada por el usuario debe cumplir dos requisitos: tener al menos 8 caracteres y ser alfanumérica (contener al menos una letra y al menos un número). Si la contraseña no cumple con alguno de los requisitos, se debe informar al usuario.
- **Caso 2:** Se utiliza una función que calcula la suma de los elementos de un generador utilizando la función `sum()` de Python. Considere que no existe certeza respecto al tipo de datos generados.
- **Caso 3:** Una función recibe la ruta de un archivo como parámetro y debe procesar su contenido. Si el archivo no puede ser procesado, el usuario debe ser informado de la situación.

Parte B — Análisis de código (3 puntos)

A partir del siguiente código, escribe el **output** que se imprime por consola al ejecutarlo. Escribe una línea por cada `print` ejecutado, respetando el orden de aparición y el texto. No se requiere justificación.

```
1 def validar(valor):
2     if not isinstance(valor, int):
3         raise TypeError(f"Debe ser un número")
4     if valor < 0:
5         raise ValueError(f"No puede ser negativo")
6     return valor
7
8 def aplicar_descuento(monto, descuento):
9     try:
10         monto = validar(monto)
11         descuento = validar(descuento)
12         print(f"Precio final: {monto - descuento}")
13     finally:
14         print("Cierre de operación")
15
16 def procesar_transaccion(monto, descuento):
17     try:
18         aplicar_descuento(monto, descuento)
19     except TypeError as error:
20         print(f"TypeError: {error}")
21     except ValueError as error:
22         print(f"ValueError: {error}")
23     else:
24         print("Reporte generado")
25
26 transacciones = [(100, 20), (-40, 10), ("80", 10)]
27 for monto, descuento in transacciones:
28     procesar_transaccion(monto, descuento)
```


Pregunta 3. [20 % nota final]

El “UNO Solitario” es un juego de cartas de una persona donde el jugador debe descartar todas las cartas de su mano en un pozo. Para descartar las cartas se debe emparejar –ya sea por número o color– la carta que está más arriba del pozo con la carta descartada. El jugador deberá realizar esta acción hasta que ya no le queden cartas en la mano o que las cartas que queden no puedan ser jugadas.

Considerando que las cartas del mazo solo contiene cartas numéricas, deberás utilizar correctamente el patrón “**Iterable Iterador**” para implementar la forma en que se juega las cartas una mano de “UNO Solitario”. Para lograr lo anterior, se representará la **Mano de Cartas** como una **Lista Ligada** de **Cartas**. Esto se ve reflejado en el siguiente código base:

```
1  from copy import deepcopy
2
3  class Carta:
4      def __init__(self, color: str, numero: int) -> None:
5          self.color = color
6          self.numero = numero
7          self.siguiente = None
8
9  class ManoCartas:
10     def __init__(self) -> None:
11         self.cabeza = None
12         self cola = None
13
14     def agregar_carta(self, carta: Carta) -> None:
15         if self.cabeza is None:
16             self.cabeza = carta
17             self.col = carta
18             return
19         self.col.siguiente = carta
20         self.col = carta
21
22     def __iter__(self) -> IteradorManoCartas:
23         return IteradorManoCartas(deepcopy(self.cabeza))
24
25 class IteradorManoCartas:
26     def __init__(self, cabeza: Carta) -> None:
27         self.mano = cabeza
28
29 if __name__ == '__main__':
30     # Se define previamente 'mano_cartas' como una instancia de ManoCartas y
31     # se le agregan múltiples instancias de Carta
32     for carta in mano_cartas:
33         print('Carta jugada:', carta.numero, carta.color)
```

Utilizando únicamente la Lista Ligada como estructura de datos, deberás completar el código anterior para permitir que se pueda iterar una instancia de **ManoCartas** asegurando que se cumpla todo lo indicado anteriormente.

Puedes utilizar pseudo-código. Además, puedes agregar nuevos atributos, crear nuevos métodos, clases y funciones siempre y cuando esto no modifique el funcionamiento de lo que ya se encuentra implementado.

Pregunta 4. [20 % nota final]

El DCCompleto 🥪 se compone de tres ingredientes: *"pan"*, *"salchicha"* y *"DCComponenteSecreto"* ingresados en ese orden. La función generadora a continuación representa cómo se procesa el último ingrediente para proseguir con lo que se lleva del DCCompleto actual, entregando 1 si se completa un DCCompleto y 0 en caso contrario. Cabe mencionar que si un ingrediente no aporta al DCCompleto, se considera que se pierde el ingrediente.

```
1 def cadena_generadora():
2     paso_actual = 0
3     elemento = yield
4     while True:
5         if elemento == "pan":
6             if paso_actual == 0:
7                 paso_actual = 1
8         elif elemento == "salchicha":
9             if paso_actual == 1:
10                paso_actual = 2
11        elif elemento == "DCComponenteSecreto":
12            if paso_actual == 2:
13                paso_actual = 0
14                elemento = yield 1
15                continue
16        elemento = yield 0
```

A continuación se te presentan tres partes **independientes** sobre la situación expuesta:

Parte A.

Dado que se entrega al generador recién instanciado de `cadena_generadora` los valores:

`(None, "salchicha", "pan", "arroz", "salchicha", "DCComponenteSecreto", "pan")`

Indicar los valores de `paso_actual` y `elemento` que la función generadora toma para cada valor entregado, considerando que llega hasta la espera de cada siguiente valor. En otras palabras, el valor que tienen las dos variables cuando se espera el siguiente valor.

Parte B.

Se te entrega un generador `gen_ingredientes_dccompleto`, que entrega de a uno un ingrediente aleatorio de los tres posibles hasta que se consume el generador.

Tu objetivo es armar un código que muestre en consola la cantidad final de DCCompletos que se arman usando `cadena_generadora` en `gen_ingredientes_dccompleto`.

Parte C.

En esta parte se evaluará el conocimiento de programación funcional, por lo que no se permite usar loops (`for`, `while`, etc.) ni estructuras de datos (`list`, `tuple`, etc.), ni `class`. Las funciones de programación funcional que puedes usar son `map`, `filter`, `reduce`, `zip`. Adicionalmente, puedes usar *lambda functions*.

Suponga que existe el generador `resultados_cadena` que te entrega los valores que va entregando un uso de `cadena_generadora` (por lo que entrega una cantidad variable de valores 0 y/o 1), deberás escribir el código que muestre la cantidad final de ingredientes perdidos.